

# I. Overview

The project implements an autonomous agent in Unity that navigates a three-dimensional terrain while avoiding a dynamically rising water level. The agent's objective is to reach a fixed goal position without ever entering a flooded region. Because the water level changes continuously over time, path validity is not static: a route that is safe when planned may become unsafe before the agent completes traversal. The system therefore performs navigation using predictive safety checks rather than relying solely on the current environment state.

All agent logic is contained within a single `MonoBehaviour`, named `AgentController`, which coordinates movement, path planning, and state transitions. The environment is represented by a Unity `Terrain`, over which a navigation graph is built. Each node in the graph corresponds to a sampled position on the terrain surface, and edges represent walkable connections between neighboring nodes. A separate `WaterController` provides the current water level as well as a time-based prediction function used to estimate future flooding.

At runtime, the agent begins at a predefined start node and attempts to reach a predefined goal node. Movement is driven by an A\* pathfinding algorithm operating on the navigation graph. Unlike standard implementations, the agent does not assume that a computed path remains valid indefinitely. Instead, it periodically evaluates whether upcoming nodes and the terrain between them will remain above the predicted water level at the time the agent is expected to reach them. If a future portion of the path is determined to become unsafe, the agent interrupts its current traversal and alters its behavior accordingly.

High-level behavior is managed through a finite state machine (FSM). The FSM separates goal-directed traversal from emergency retreat and waiting behavior, allowing the agent to respond coherently to changes in water level. When the tide is not threatening, the agent moves directly toward the goal. When future flooding is detected, it abandons or truncates the current path and attempts to reach nearby high ground instead. If no immediate progress toward the goal is possible, the agent remains on safe terrain while periodically re-evaluating the goal's reachability.

Movement is performed directly in world space but constrained to the terrain surface. The agent continuously samples terrain height and normals to remain aligned with slopes and maintain contact with the ground. Movement speed is adjusted based on elevation changes to simulate reduced uphill traversal speed. The agent's vertical position is explicitly corrected every frame to ensure it remains above the terrain surface.

The system is designed to minimize unnecessary recomputation. Pathfinding and safety analysis are not executed every frame but are triggered by state transitions or performed at fixed time intervals. When paths are replanned, continuity is preserved whenever possible by

selecting the closest forward-compatible point on the new path instead of restarting from the beginning. This prevents visible backtracking and maintains smooth movement.

The following chapters describe the internal components of the system in detail. Chapter 2 focuses on the finite state machine and its transitions. Chapter 3 describes the navigation graph and pathfinding strategy. Chapter 4 presents the predictive tide safety analysis. Subsequent chapters discuss movement, replanning logic, and robustness considerations.

---

## 2. Finite State Machine

The agent's high-level behavior is controlled through an explicit finite state machine implemented within the `AgentController`. The FSM is responsible for deciding what the agent should do at any given time, while lower-level systems such as pathfinding and movement are responsible for how those actions are executed. This separation simplifies reasoning about the agent's behavior and prevents navigation logic from becoming tightly coupled to individual movement conditions.

The finite state machine is initialized during the `Start()` method, after the terrain graph and water controller have been successfully retrieved. Each state is defined by a set of actions executed on entry and during updates, as well as a set of transitions governed by boolean conditions. The FSM implementation used in the project supports explicit enter actions and stay actions, allowing path planning and traversal logic to be clearly separated.

By structuring behavior around an explicit finite state machine, the system ensures that navigation decisions remain coherent even under rapidly changing environmental conditions. Each state has a clearly defined responsibility, and transitions are driven by measurable conditions derived from the agent's predictive safety checks. This approach allows the agent to balance progress and survival without resorting to ad hoc conditional logic distributed across the codebase.

### 2.1 Traverse state

The default and initial state of the agent is the traversal state. In this state, the agent attempts to reach the goal node by following a path computed using A\* pathfinding. Upon entering the traversal state, the agent calls a path planning function that computes a route from the agent's current node to the goal. If a valid path is found, the agent begins moving along it. Movement itself is handled separately and continues as long as the agent remains in the traversal or retreat-related states.

While traversing toward the goal, the agent continuously evaluates whether the current path remains safe. This evaluation does not simply check the agent's current position, but instead predicts whether future nodes and edges along the path will become flooded before the agent

reaches them. If such a condition is detected, the FSM triggers a transition out of the traversal state into a retreat-oriented state that prioritizes survival over progress.

## **2.2 Seek High Ground state**

The seek-high-ground state is entered when the agent determines that its current path will become unsafe due to rising water. Upon entering this state, the agent attempts to preserve as much progress as possible. If the predictive safety analysis identified a future node that remains above the maximum expected water level, the current path is truncated so that the agent continues moving forward to that safe point instead of abandoning the entire route. Only when no such safe continuation exists does the agent compute a new path toward the nearest reachable high-ground node.

While in the seek-high-ground state, movement proceeds in the same way as during normal traversal. The difference lies in the transition logic. As long as the water level is rising, or the goal cannot be reached safely, the agent remains in this state. When the water stops rising and a safe path to the goal becomes available again, the FSM transitions the agent back into the traversal state, restoring goal-oriented behavior without resetting the agent's position.

## **2.3 Wandering state**

If the agent reaches a safe elevated position but is unable to make further progress toward the goal, the FSM transitions into a wandering state. This state represents a holding behavior rather than active goal pursuit. The agent remains on safe terrain and periodically attempts to find a partial or complete path toward the goal. This prevents the agent from continuously recomputing unreachable paths while still allowing recovery if conditions improve.

In the wandering state, the agent alternates between short movements across adjacent safe nodes and periodic reachability checks. If a path toward the goal becomes safely reachable again, the FSM immediately transitions back to the traversal state. Otherwise, the agent continues to reposition locally to avoid stagnation and maintain plausible behavior while waiting for the environment to change.

## **2.4 Goal Reached state**

Once the agent reaches the goal node, it transitions into a terminal goal-reached state. In this state, movement is disabled and no further transitions are evaluated. This cleanly terminates execution and prevents unnecessary updates after the objective has been achieved.

---

# **3. Terrain Graph and Pathfinding**

Navigation in the system is performed over an explicit graph representation derived from the Unity terrain. A custom graph is used in order to retain full control over node placement, edge

connectivity, and cost evaluation. This choice is motivated by the need to tightly integrate pathfinding with terrain elevation and time-based safety checks.

### 3.1 Terrain Graph

The terrain is discretized into a two-dimensional grid aligned with the terrain surface. Each grid cell corresponds to a node in the navigation graph and stores its position in world space, including the sampled terrain height at that location. Nodes are arranged in a regular matrix structure, which allows fast lookup based on world coordinates when querying the agent's current position. Edges are created between neighboring nodes to represent walkable connections across the terrain surface.

Connections between nodes are local and spatially constrained. Each node is connected only to a limited set of adjacent nodes, ensuring that paths follow the terrain topology without introducing unrealistic shortcuts. The resulting graph is dense enough to capture terrain variations while remaining manageable for repeated A\* searches at runtime.

Each edge is assigned a cost based on the height difference between the outgoing node and the destination node, reflecting slower movement and increased exposure time. In addition, each edge gets a danger penalty if it leads to a potentially unsafe node, the penalty is proportionate to the height difference between start node and destination node of the edge.

### 3.2 Pathfinding

Pathfinding is implemented using the A\* algorithm, operating directly on the terrain graph. The algorithm uses a custom heuristic function that combines Euclidean distance to the goal with a bias toward higher terrain. This bias is introduced by subtracting a term proportional to the node's height from the standard Euclidean distance. As a result, paths that traverse higher ground are naturally favored when their overall distance is comparable.

By preferring elevated areas, the agent indirectly increases its safety margin against future flooding, even before predictive checks are applied. This reduces the frequency of emergency retreats and produces paths that are more robust to rising water levels without explicitly encoding water information into the path cost.

### 3.3 Planning

Once a path has been computed, it is stored as an ordered list of nodes. The agent advances through this list incrementally, maintaining an index that identifies the current target node. Pathfinding is not performed every frame; instead, the current path is reused as long as it remains safe and valid. New paths are computed only when entering certain FSM states or when the safety analysis determines that the existing path will become unsafe.

The system also addresses a common issue in dynamic replanning scenarios: loss of progress due to path recomputation. When a new path is generated, the agent does not automatically restart from the first node. Instead, it evaluates whether the new path requires backtracking or whether it overlaps with the agent's current direction of movement. If overlap exists, the

agent selects the closest forward-compatible node on the new path and continues from there. This preserves momentum and prevents oscillatory behavior caused by frequent replanning.

By combining a custom terrain graph with a height-aware heuristic and careful path reuse logic, the pathfinding system provides stable and predictable navigation while remaining flexible enough to support dynamic environmental constraints. The next chapter focuses on how these paths are evaluated over time to determine whether they remain safe in the presence of rising water.

---

## 4. Predictive Tide Safety Analysis

A central aspect of the system is the ability to determine whether a planned path will remain traversable in the future, rather than whether it is safe at the moment of evaluation. Because the water level changes continuously over time, a static comparison between terrain height and the current water level is insufficient. The agent must instead estimate where it will be at a given time and compare that position against the predicted water level at that same time.

This analysis is performed inside the FSM transition logic, primarily when deciding whether it is safe to continue traversing the current path or whether the agent must switch to a retreat-oriented state. The safety evaluation is therefore tightly coupled to decision-making rather than movement execution, ensuring that expensive predictive checks are performed only when needed.

The first step of the analysis consists of estimating the time required to reach future nodes along the current path. Starting from the agent's current path index, the system iterates forward through a limited number of upcoming nodes. For each edge between consecutive nodes, the traversal time is computed by dividing the spatial distance by the expected movement speed. Once an estimated arrival time is available, the water controller is queried to determine the predicted water level at that time. A node is considered unsafe if its height is lower than the predicted water level plus a configurable safety margin. If such a node is detected within the lookahead window, the current path is considered invalid and a state transition is triggered.

While this node-based check is necessary, it is not sufficient on its own. The terrain between two nodes may dip below the terrain height of the nodes themselves, creating a situation where both endpoints are safe but the path between them becomes flooded. To address this, the system performs additional sampling along each edge of the path.

For each edge under evaluation, a set of intermediate sample points is generated by interpolating between the two node positions. The number of samples depends on the physical length of the edge, ensuring that longer edges are checked more thoroughly without oversampling short connections. Each sample point is projected onto the terrain to obtain the actual ground height at that location. Using the same traversal-time estimation logic, the

system computes the approximate time at which the agent would reach each sampled position and queries the water controller for the predicted water level at that time.

If any sampled point along an edge is predicted to fall below the water level, the entire path segment is marked as unsafe. This prevents the agent from traversing valleys or depressions that would become submerged before the agent reaches the next node.

During this predictive scan, the system also tracks the furthest node ahead that remains safely above the maximum expected water level. This information is used to optimize retreat behavior. Instead of immediately abandoning the current path when an unsafe region is detected, the agent can continue moving forward to this furthest safe node. This allows the agent to make incremental progress toward elevated terrain even under rising tide conditions, rather than repeatedly retreating to earlier positions.

The predictive safety analysis is deliberately limited in scope for performance reasons. Only a fixed number of nodes ahead of the current position are examined, and the checks are executed only when evaluating FSM transitions or at controlled time intervals.

By combining arrival-time estimation, node-based safety checks, and edge sampling, the system achieves a robust form of temporal path validation. Paths are evaluated not just in space, but in space and time, allowing the agent to anticipate danger and react before it becomes trapped. This mechanism forms the foundation for the agent's ability to survive in a continuously changing environment and enables the higher-level behaviors described in the previous chapters.

---

## 5. Movement and Terrain Alignment

The movement system is responsible for executing the navigation decisions produced by the finite state machine and the pathfinding subsystem. Its role is not to make high-level decisions, but to move the agent smoothly and correctly across the terrain surface while respecting physical constraints such as slope orientation, terrain height, and water level.

Movement is performed in world space toward a target position derived from the current node in the active path. The agent maintains an index into the current path, and the corresponding node position is used as the next movement target. To avoid abrupt vertical motion and to maintain consistent contact with the terrain, the target position is adjusted by adding half of the agent's collider height to the sampled terrain height. This ensures that the agent remains fully above the ground and avoids interpenetration with the terrain mesh.

Each update step computes a direction vector from the agent's current position toward the adjusted target. This direction is normalized and used to advance the agent forward by a distance proportional to the selected movement speed and frame time. Movement speed is not constant: when the agent is moving uphill, detected by comparing the vertical difference

between the current position and the target, a reduced speed is used. Flat or downhill movement uses a higher base speed. This distinction directly affects traversal time estimation and therefore impacts the predictive safety analysis described in the previous chapter.

While moving, the agent continuously corrects its vertical position by sampling the terrain height below it. The terrain sampling is performed using Unity's optimized `SampleHeight` function, and the agent's world-space Y coordinate is adjusted accordingly every frame. This explicit correction ensures that the agent remains aligned to the terrain surface even when moving across uneven regions or when numerical drift would otherwise cause it to float above or sink into the ground.

Orientation is handled separately from translation. To align the agent with the terrain slope, the system computes the interpolated surface normal at the agent's current position using Unity's terrain data. This is achieved by converting the agent's world position into normalized terrain coordinates and querying the terrain's normal map. The resulting normal vector defines the agent's local up direction.

A forward direction is derived by projecting the movement direction onto the terrain surface. This is done through cross products between the movement direction and the surface normal, producing a basis that respects the local slope while preserving forward motion. The agent's rotation is then smoothly interpolated toward this orientation using spherical interpolation, avoiding abrupt changes in rotation and resulting in visually stable motion over steep or irregular surfaces.

Node transitions are handled using a distance threshold in the horizontal plane. When the agent's projected position in the XZ plane is sufficiently close to the current target node, the path index is advanced and the next target is selected. This avoids reliance on exact position matching, which would be unreliable due to continuous terrain corrections and floating-point error.

Although movement is updated every frame, it remains a purely reactive process driven by the current path and FSM state. If movement is disabled due to a state transition or path exhaustion, the system immediately stops advancing the agent without requiring additional conditions. This clear separation between movement execution and behavioral decision-making simplifies debugging and ensures that complex navigation logic does not interfere with low-level motion.

Through constant terrain sampling, slope-aware orientation, and adaptive speed control, the movement system ensures that path execution remains physically plausible while remaining fully compatible with the predictive navigation logic. This allows the agent to traverse complex terrain reliably while reacting appropriately to environmental changes.

---

## 6. Path Continuity and Replanning Logic

Whenever a new path is computed, either toward the goal or toward a high-ground destination, the agent evaluates how this path relates to the one it was previously following. If the agent is already in motion, it is likely closer to an intermediate node than to the start of the new path. Restarting from the first node in such cases would cause the agent to backtrack unnecessarily, even when the new path overlaps with the previous one.

To avoid this, the system first determines whether backtracking is unavoidable. This is done by comparing the forward direction of the current path with the initial direction of the newly computed path. The forward direction of the current path is obtained from the vector between the agent's current node and the next node along the path. The initial direction of the new path is computed from its first two nodes. If the dot product between these vectors is negative, the paths diverge in opposite directions and backtracking is required. In this case, the agent resets its path index and begins following the new path from its start.

If backtracking is not required, the agent attempts to continue moving forward along the new path. The system searches for the closest node on the new path that is spatially near the agent's current position and lies ahead along the movement direction. This search is limited to the first few nodes of the path to reduce computational cost and to avoid selecting nodes that would cause sudden jumps in direction. The node with the smallest distance to the agent is selected as the new target, and traversal continues from that point.

In addition to preserving continuity, the system distinguishes between full replanning and partial path reuse. When a future unsafe region is detected, the predictive safety analysis may identify a furthest reachable safe node along the existing path. In such cases, the agent does not compute a new path at all. Instead, the current path is truncated so that the agent continues moving toward that safe node. This strategy significantly reduces unnecessary replanning and allows the agent to exploit safe portions of previously computed paths.

Together, these mechanisms ensure that navigation remains stable and visually coherent even under frequent environmental changes. Replanning becomes an adaptive process that respects both the agent's current motion and its long-term objective, rather than a disruptive reset that sacrifices progress.

---

## 8. Conclusions and Limitations

This project implements a navigation agent capable of operating in a dynamic environment where path safety depends not only on spatial properties but also on time. By combining a finite state machine, graph-based pathfinding, predictive safety analysis, and terrain-aware movement, the agent can consistently reach its goal while avoiding rising water, adapting its behavior as environmental conditions change.

A key characteristic of the system is that navigation decisions are based on future estimates rather than current state alone. Paths are not treated as immutable plans but as tentative



trajectories that are continuously validated against predicted water levels. This allows the agent to interrupt traversal before entering unsafe regions and to recover when conditions improve, without relying on purely reactive behavior.

Several extensions were considered during development but deliberately not implemented. One example is path cutting, a common optimization used in navigation systems to reduce path length and improve movement smoothness. Path cutting typically involves connecting the current position directly to a further node on the path if there is clear line-of-sight between the two, thereby removing intermediate nodes.

While this approach can be effective in planar or two-dimensional environments, it proved unreliable in the three-dimensional terrain context used in this project. Visibility between two nodes does not guarantee that the terrain between them is traversable in a faster or safer manner. Slopes, local elevation changes, and terrain curvature can significantly affect traversal speed and arrival time. In such cases, cutting directly to a distant node may result in a path that is geometrically shorter but temporally slower due to steep uphill movement, or even unsafe when evaluated against future water levels.

Additionally, because this system relies on predictive arrival-time analysis, any shortcut introduced by path cutting would require the same level of temporal and spatial validation as the original path. Ensuring correctness in all cases would therefore require dense terrain sampling and time estimation along the cut segment, reducing or eliminating the performance benefits of the optimization.

For these reasons, the system prioritizes path correctness and predictability over aggressive path shortening. Navigation behavior remains stable, incremental, and explainable, which is particularly important in an environment where safety constraints dominate over purely geometric optimality.